Real-Time Clickstream Analytics & Financial Reconciliation Lakehouse

Table of Contents

1. Project Overview

This project is an end-to-end data engineering platform designed to process real-time clickstream events, capture transactional changes from operational databases, and perform financial reconciliation using a modern **Medallion Lakehouse Architecture.**

The system simulates an international e-commerce environment where millions of user interactions and transactions are generated daily. It combines the following disciplines into a single production-grade solution:

- Real-time event streaming with Apache Kafka
- Change Data Capture (CDC) from PostgreSQL
- Cloud data lake storage on AWS S3 with Apache Iceberg table format
- Declarative SQL transformations using dbt inside Snowflake
- End-to-end pipeline orchestration with Apache Airflow
- Business intelligence dashboards for executive and operational reporting

2. Business Problem

Traditional batch pipelines cannot provide real-time insights or maintain a complete historical audit trail. This project addresses those challenges by simulating three core enterprise problems:

Customer Analytics

- Which products are customers viewing right now?
- Which products are frequently added to carts but never purchased?
- What customer behavior patterns lead to completed purchases?
- How can product recommendations be generated in near real-time?

Financial Reconciliation

- Did every successful transaction reach the settlement system?
- Are there missing or duplicate settlements in the pipeline?
- Can finance teams identify discrepancies automatically without manual checking?

Historical Auditing

- What was a customer's segment classification at the time of a specific purchase?
- What was the product price six months ago?
- How can historical records be preserved without overwriting existing data?

3. Data Sources

The project uses **4 primary data sources** and **1 optional data source.**

#	Source Name	Format	Ingestion Mode	Description
1	Clickstream Events	JSON	Real-time (Kafka)	Every user action on the e-commerce platform — searches, views, add-to-cart, purchases
2	PostgreSQL Operational DB	Relational rows	CDC / Batch	Authoritative backend records — transactions, customer accounts, inventory updates
3	Third-Party Logistics & Shipping API	Nested JSON	Batch (AWS S3 via Lambda)	External vendor delivery data (FedEx/DHL-style), nested arrays, semi-structured
4	Server / API Gateway Logs	Raw text (space-delimited)	Real-time (Kafka)	Backend server health, request traces, error codes, performance metrics

Data Source 1: Clickstream Events (Real-Time Stream)

Every action a user performs on the platform generates an event. Together, these events form the **clickstream**. Example user journey:

```
Open Website
      ↓
Search Laptop
      ↓
View Laptop Product Page
      ↓
Add Laptop To Cart
      ↓
Purchase Laptop
```

Since this is a portfolio project, clickstream events are simulated using a **Python-based Clickstream Generator** that continuously produces realistic events and publishes them to Apache Kafka topics.

Data Source 2: PostgreSQL Operational Database

The PostgreSQL database is the **internal financial source of truth**. It records officially authorized transactions, customer account registrations, and inventory changes processed by the backend servers. Changes in this database are captured using **CDC (Change Data Capture)** so that every insert, update, and delete is tracked and propagated downstream.

Data Source 3: Third-Party Logistics & Shipping

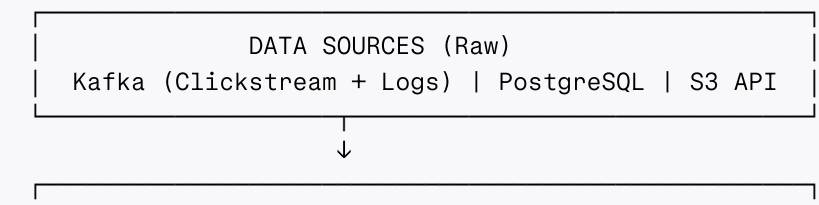
Unlike structured relational data, this source produces **semi-structured nested JSON** — the format used by real external vendor APIs (e.g., FedEx, Delhivery). This forces the pipeline to handle nested objects and arrays, flattening them into clean relational rows during the transformation phase.

Data Source 4: Server Application / API Gateway Logs

Every time a user action hits the backend (FastAPI, Spring Boot, or Nginx), the server emits a **raw, space-delimited text log line**. These logs act as a digital ledger of system health and are critical for detecting server timeouts and dropped order events.

4. Platform Architecture

The project follows the **Bronze → Silver → Gold** Medallion Architecture pattern.



Pipeline Pattern: ELT (Extract → Load → Transform)

Because the project uses Snowflake as its transformation engine, the pattern is **ELT** — not ETL:

1. **Extract & Load** raw data into the Bronze Layer (S3 / Iceberg) as-is
2. **Transform** inside Snowflake using dbt models to build Silver and Gold layers

5. Phase 1 — Data Ingestion (Bronze Layer)

Overview

With all four data sources generating realistic data, the ingestion phase extracts data from those active sources and lands it into a unified, reliable storage format — the **Bronze Layer**.

The Bronze Layer is the raw, untransformed landing zone. No business logic or cleaning is applied at this stage. Data fidelity is preserved exactly as it arrived.

Architecture: Hybrid Streaming + Batch

Layer	Sources	Technology
Streaming Layer	Clickstream (JSON) + Server Logs (Raw Text)	Apache Kafka → PySpark Structured Streaming → Iceberg
Batch Layer	PostgreSQL (CDC) + 3PL Shipping API (JSON)	AWS Lambda → S3 Landing Bucket / Kafka Connect CDC

Ingestion Steps

1. **Kafka Topic Setup** — Create separate Kafka topics for clickstream events and server logs
2. **PySpark Structured Streaming** — Read from Kafka topics in micro-batches, parse raw bytes, and write to Iceberg tables in S3
3. **PostgreSQL CDC** — Use Kafka Connect with Debezium to capture row-level changes and stream them into Kafka, then land into S3
4. **3PL Shipping API** — AWS Lambda function polls the shipping vendor API on a schedule and drops nested JSON files into the S3 landing bucket

Output of This Phase

After completing ingestion, you have:

- Raw Iceberg tables in S3 for streaming data (clickstream, server logs)
- Parquet / JSON files in S3 landing buckets for batch data (PostgreSQL CDC, shipping API)

6. Phase 2 — Transformations (Silver Layer)

Overview

The Silver Layer is the **"T" in ELT** — this is where raw Bronze data is cleaned, unified, and enriched into analytics-ready tables. The technology stack introduced here is the **Snowflake + dbt (Data Build Tool)** combination.

Step 1: Connect S3 / Iceberg to Snowflake

- For **batch data on S3**: Create Snowflake **External Tables** or use **SnowPipe** to auto-ingest Parquet and JSON files into Snowflake staging tables.
- For **Iceberg tables**: Connect Snowflake directly to the Iceberg catalog for seamless querying.

Step 2: Initialize dbt

Instead of writing manual SQL scripts, **dbt** manages all transformations as modular, version-controlled `SELECT` statements. Benefits:

- Every transformation is a plain `.sql` file tracked in Git
- dbt compiles and executes those SQL models inside Snowflake
- Built-in testing, documentation, and lineage graph

Step 3: Write dbt Silver Models

Each Silver model performs three critical engineering tasks:

Task	What It Does	SQL Technique
Data Cleaning & Type Casting	Convert strings to timestamps, cast IDs to integers, handle nulls	`CAST()`, `COALESCE()`, `TO_TIMESTAMP()`
Flatten Semi-Structured Data	Break nested JSON arrays into flat rows and columns	Snowflake `FLATTEN()`, `:` operator
Deduplication	Remove duplicate streaming events caused by network retries	`ROW_NUMBER() OVER (PARTITION BY ...)`

Silver Layer Output Tables

Table	Source	Description
`silver_users`	PostgreSQL CDC	Cleaned customer account records
`silver_products`	PostgreSQL CDC	Cleaned product catalog
`silver_orders`	PostgreSQL CDC	Cleaned order transactions
`silver_shipping_status`	3PL Shipping API (JSON)	Flattened delivery status records
`silver_clickstream` ⭑	Kafka / Iceberg	Cleaned and deduplicated user events
`silver_server_logs` ⭑	Kafka / Iceberg	Cleaned and type-cast backend log records

7. Phase 3 — Gold Layer (Analytical Marts)

Overview

The Gold Layer stops treating data as six separate tables and starts **combining them to solve core business problems.** Using dbt, highly optimized and aggregated tables (**Data Marts**) are built to answer executive-level questions and power the portfolio's key use cases.

Gold Model 1: Data Reconciliation Mart (`gold_order_reconciliation`)

The star of the portfolio — catches pipeline errors and financial discrepancies.

- **Logic:** Performs a `FULL OUTER JOIN` between `silver_orders` (backend database) and the streaming `silver_clickstream` + `silver_server_logs` tables using `session_id`
- **Output:** Calculated boolean flags including:
  - `is_paid_but_missing_in_db` — payment processed but order never written to DB
  - `is_server_timeout_drop` — backend server timeout killed the order mid-submission
- **Business Value:** Surfaces the exact **50 missing records** where a backend server timeout crashed an order as the user clicked submit

Gold Model 2: Logistics Performance Mart (`gold_shipping_performance`)

Measures the operational efficiency of third-party shipping vendors.

- **Logic:** Joins `silver_orders` with `silver_shipping_status` using `order_id`
- **Output:** Calculated metrics including:
  - `delivery_latency_hours` — exact hours from `order_placed_at` to `status_last_updated_at`
  - `average_delivery_latency` by carrier
  - Flags for carriers breaching SLA thresholds
- **Business Value:** Provides data-driven evidence to hold shipping carriers accountable for delays

Gold Model 3: Customer 360 Dimension (`gold_dim_customers`)

A comprehensive, single-view table of user behavior, combining identity, transactions, and web traffic.

- **Logic:** Joins `silver_users` with aggregated metrics from `silver_orders` and `silver_clickstream`, grouped by `user_id`
- **Output:** High-level user profiles including:
  - `total_lifetime_spend`
  - `favorite_product_category`
  - `last_active_date`
  - `total_orders_placed`
- **Business Value:** Powers personalization engines, customer segmentation, and marketing campaigns

8. Phase 4 — Pipeline Orchestration (Airflow)

Overview

Apache Airflow shifts the project from a collection of isolated scripts into a **fully automated, production-grade data platform**. Airflow acts as the central brain (orchestrator), ensuring data flows seamlessly, dependencies are respected, and failures are immediately caught and reported.

What is an Airflow DAG?

A **DAG (Directed Acyclic Graph)** is a Python file that defines the pipeline:

- **Directed** — Data flows in one direction: Ingestion → Silver → Gold
- **Acyclic** — The pipeline cannot loop back on itself; it has a clear start and finish

- **Graph** — A structural map showing how tasks connect to each other

Core Benefits of Orchestration

Benefit	What It Solves
Dependency Management	Task B cannot start until Task A finishes successfully
Error Handling & Alerting	Auto-catches failures and sends Slack/email alerts to engineers
Idempotency & Retries	Automatically retries failed tasks up to N times with configurable delay
Scheduling	Runs the entire pipeline daily at midnight without human intervention

The 6-Task DAG Pipeline

```
start_pipeline
      ↓
ingest_batch_and_stream    (PySpark → Bronze Layer)
      ↓
run_silver_cleanse         (dbt run --select tag:silver)
      ↓
run_gold_marts             (dbt run --select tag:gold)
      ↓
run_data_quality_tests     (dbt test)
      ↓
end_pipeline
```

Task	Operator	Operation
`start_pipeline`	EmptyOperator	Initializes the run, logs execution timestamp
`ingest_batch_and_stream`	BashOperator	Runs PySpark to land all raw data into Bronze Layer
`run_silver_cleanse`	BashOperator	Executes `dbt run --select path:models/staging`
`run_gold_marts`	BashOperator	Executes `dbt run --select path:models/marts`
`run_data_quality_tests`	BashOperator	Executes `dbt test` for data integrity assertions
`end_pipeline`	EmptyOperator	Closes the run, clears temporary memory

Airflow DAG Code Blueprint

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.empty import EmptyOperator
from airflow.operators.bash import BashOperator

default_args = {
    'owner': 'your_name',
    'depends_on_past': False,
    'start_date': datetime(2026, 1, 1),
    'email_on_failure': True,
```

```
    'email': 'your_email@example.com',
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    dag_id='lakehouse_medallion_pipeline',
    default_args=default_args,
    description='Automated orchestration from Ingestion to Gold reconciliation marts
    schedule_interval='@daily',
    catchup=False,
    tags=['pyspark', 'dbt', 'snowflake'],
) as dag:

    start_pipeline = EmptyOperator(task_id='start_pipeline')

    ingest_bronze = BashOperator(
        task_id='ingest_batch_and_stream',
        bash_command='spark-submit --master local[*] /path/to/pyspark_ingestion_scri
    )

    run_silver_layer = BashOperator(
        task_id='run_silver_cleanse',
        bash_command='cd /path/to/dbt/project && dbt run --select path:models/staging
    )

    run_gold_layer = BashOperator(
        task_id='run_gold_marts',
        bash_command='cd /path/to/dbt/project && dbt run --select path:models/marts'
    )

    test_data_integrity = BashOperator(
        task_id='run_data_quality_tests',
        bash_command='cd /path/to/dbt/project && dbt test'
    )

    end_pipeline = EmptyOperator(task_id='end_pipeline')

    # Dependency chain
    start_pipeline >> ingest_bronze >> run_silver_layer >> run_gold_layer >> test_da
```

9. Phase 5 — Visualization & BI Dashboard

Overview

The Business Intelligence phase converts all the data engineering work into **direct, actionable business value.** Executives, product managers, and operations teams need clean, interactive dashboards — not raw SQL queries. This phase connects the Gold Layer in Snowflake to a BI tool (Power BI or Tableau) to build an enterprise-grade **Data Integrity & Logistics Operations Dashboard.**

Dashboard Structure

The dashboard is divided into two functional zones:

Zone A: Revenue Leakage & Pipeline Integrity Monitor

Data source: `gold_order_reconciliation`

Component	Type	What It Shows
Missing Orders Alert	KPI Metric Card	Count of orders where `is_server_timeout_drop = TRUE` (e.g., 50 missing orders)
Financial Exposure Tracker	KPI Metric Card	`SUM(total_order_amount_usd)` for all dropped orders — exact dollar impact
Discrepancy Trend	Time-Series Line Chart	When the 50 drops occurred over the last 30 days — identifies spike timestamps

Zone B: Logistics & Carrier Performance Analytics

Data source: `gold_shipping_performance`

Component	Type	What It Shows
Avg Delivery Latency	Horizontal Bar Chart	Average delivery hours grouped by `shipping_carrier` (FedEx vs. DHL vs. Blue Dart)
SLA Breach Rate	Pie Chart	% of packages flagged `LATE DELIVERY` vs. `ON TIME`
Live Shipment Exceptions	Table / Ledger	Specific `order_id` rows currently flagged `DELAYED` for customer support teams

Technical Implementation Workflow

1. **Warehouse Hookup** — Open Power BI / Tableau, select the native **Snowflake Connector**, and point it to the `GOLD_LAYER` database schema
2. **Connection Mode Selection:**
   - **Logistics Mart** → `Import / Scheduled Refresh` (data updated 4× per day — shipping changes slowly)
   - **Reconciliation Mart** → `DirectQuery` mode (live SQL sent to Snowflake every time a manager opens the dashboard)
3. **Publishing & Distribution** — Publish to Power BI Service / Tableau Cloud, configure automated daily email snapshots to the engineering lead every morning

10. Tech Stack Summary

Category	Technology	Purpose
Event Streaming	Apache Kafka	Ingesting real-time clickstream and server log events
Stream Processing	PySpark Structured Streaming	Reading Kafka topics and writing to Iceberg in micro-batches
CDC	Debezium + Kafka Connect	Capturing row-level PostgreSQL changes
Object Storage	AWS S3	Central data lake storage for all layers
Table Format	Apache Iceberg	ACID-compliant, schema-evolving table format on S3
Data Warehouse	Snowflake	SQL execution engine for Silver and Gold transformations
Transformation Tool	dbt (Data Build Tool)	Modular, version-controlled SQL models
Orchestration	Apache Airflow	Scheduling, dependency management, alerting
Visualization	Power BI / Tableau	Executive dashboards connected to Snowflake
Infrastructure	Docker / Docker Compose	Local containerized development environment
Cloud Platform	AWS (S3, Glue, IAM, Lambda)	Cloud-native storage and serverless functions
Language	Python, SQL, YAML	Pipeline code, transformations, configuration
Operational DB	PostgreSQL (Aiven)	Managed external PostgreSQL hosting

> **Project Status:** End-to-end platform architecture complete — covering Data Ingestion → Bronze → Silver → Gold → Orchestration → Visualization.